

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)

ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : P.Neelima

Reg. No 11249A276

Class: IIB.E.(CSE)

Course Code:

Course Name: Data Science Toolkit Lab

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)

ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done
by P.Neelima with **Reg. No**
11249A276 of II-B.E.(CSE) during the academic year 2025 –2026 .

Station: **ENATHUR**

Date:

Staff-in-charge
Department

Head of the

Submitted for the Practical examination held on:

Examiner-1

Examiner-2

INDEX

SL.NO	Program Name	Page no	Date	Sign
1.1	NUMPY	3-8	22-1-26	
1.2	PANDAS	9-11	29-1-26	
1.3	MATPLOTLIB	12-15	5-2-26	
1.4	SCIKIT-LEARN	16-19	12-2-26	
2	Perform Data Exploration and preprocessing in Python	20-24	12-2-26	
3	Implement Regularised Linear Regression	25-26	19-2-26	
4	Implement Naive Bayes Classifier	27-29	5-3-26	
5	Implement Regularised Logistic Regression	30-33	5-3-26	
6	Build models using different Ensembling Techniques	34-36	12-3-26	
7	Build models using decision Trees	37-38	19-3-26	
8	Support Vector Machine (SVM) with different kernels	39-41	3-4-26	
9	Implement K-Nearest Neighbors (KNN) Classification	42-44	3-4-26	
10	K-means clustering with PCA and ELBOW method	45-48	10-4-26	

EX-01

1.NUMPY(LIBRARY)

```
import numpy as np
```

```
# Create a list and convert to NumPy array
```

```
list1 = [1, 2, 3]
```

```
vector1 = np.array(list1)
```

```
print("Vector 1:", vector1)
```

```
# Create a range array
```

```
vector = np.arange(1, 6)
```

```
print("Arange vector:", vector)
```

```
# Create zeros array
```

```
vector = np.zeros(5)
```

```
print("Zeros:", vector)
```

```
# Dot product example
```

```
list1 = [5, 1, 1]
```

```
list2 = [3, 2, 1]
```

```
vector1 = np.array(list1)
```

```
vector2 = np.array(list2)
```

```
print("Vector 1:", vector1)
```

```
print("Vector 2:", vector2)
```

```
product = vector1.dot(vector2)
```

```
print("Dot product:", product)
```

```
# More NumPy examples
```

```
a = np.array([1, 2, 3, 4, 5])
```

```
b = np.arange(1, 10, 2)
```

```
c = np.linspace(0, 1, 5)
```

```
d = np.zeros((2, 3))
```

```
e = np.ones((2, 2))
```

```
f = np.eye(3)
```

```
g = np.random.randint(1, 100, size=(3, 3))
```

```
print("a:", a)
```

```
print("b:", b)
```

```
print("c:", c)
```

```
print("d:\n", d)
```

```
print("e:\n", e)
```

```
print("f (identity matrix):\n", f)
```

```
print("g (random):\n", g)
```

2)Array properties

```
print("Shape:",g.shape)
print("Dimensions:", g.ndim)
print("Size:", g.size)
print("Data type:", g.dtype)
```

3) Reshape & Transpose

```
reshaped=g.reshape(1,12)
print("Reshaped array:\n", reshaped)
print("Transpose:\n", g.T)
print("Flattened:", g.flatten())
```

4) Mathematical Operations

```
print("Add:", np.add(a,2))
print("Multiply:", np.multiply(a, 3))
print("Square root:", np.sqrt(a))
print("Power:", np.power(a, 2))
print("Absolute:", np.abs([-2, 3]))
```

5) Statistical Functions

```
print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Standard Deviation:", np.std(a))
print("Variance:", np.var(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
print("Sum:", np.sum(a))
print("Percentile (50):", np.percentile(a, 50))
```

6)Axis-based operations

```
print("Column-wisesum:\n", np.sum(g, axis=0))
print("Row-wise mean:\n", np.mean(g, axis=1))
```

7) Indexing & Slicing

```
print("First element:", a[0])
print("Slice:", a[1:4])
print("2D Slice:\n", g[0:2, 1:3])
```

8)Boolean & Conditional Operations

```
print("Where > 50:\n", np.where(g >50))
print("Any > 90:", np.any(g > 90))
print("All > 0:", np.all(g > 0))
```

10)Linear Algebra

```
x = np.array([[1, 2],[3, 4]])
y = np.array([[5, 6],[7, 8]])
print("Dot Product:\n", np.dot(x, y))
print("Matrix Multiplication:\n", np.matmul(x, y))
print("Determinant:", np.linalg.det(x))
print("Inverse:\n", np.linalg.inv(x))
```

11)Random Functi

```
import numpy as np
np.random.seed(1)
a = np.array([1, 2, 3, 4, 5])
rand_sample = np.random.choice(a, size=3)
print("Random choice:", rand_sample)
```

12)# Create arrays

```
h1 = np.array([1, 2, 3])
h2 = np.array([4, 5, 6])
```

Concatenate and Stack

```
print("Concatenate:", np.concatenate((h1, h2)))
print("Vertical Stack:\n", np.vstack((h1, h2)))
print("Horizontal Stack:", np.hstack((h1, h2)))
import numpy as np
```

13)# Save array to file

```
np.save("sample-array.npy", a)
# Load array from file
loaded = np.load("sample-array.npy")
print("Loaded array:", loaded)
```

colab.research.google.com

11249A276 - Colab

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[5]: ... Vector 1: [1 2 3]
      Arange vector: [1 2 3 4 5]
      Zeros: [0. 0. 0. 0. 0.]
      Vector 1: [5 1 1]
      Vector 2: [3 2 1]
      Dot product: 18
      a: [1 2 3 4 5]
      b: [1 3 5 7 9]
      c: [0. 0.25 0.5 0.75 1. ]
      d:
      [[0. 0. 0.]
       [0. 0. 0.]]
      e:
      [[1. 1.]
       [1. 1.]]
      f (identity matrix):
      [[1. 0. 0.]
       [0. 1. 0.]
       [0. 0. 1.]]
      g (random):
      [[ 7 26 51]
       [21 19 85]
       [12 29 30]]
      Shape: (3, 3)
      Dimensions: 2
      Size: 9
      Data type: int64
      Reshaped array:
      [[ 7]
       [26]
       [51]
       [21]
       [19]
       [85]]
```

Variables Terminal

6:40 PM Python 3

colab.research.google.com

11249A276 - Colab

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[5]: ... [[ 7]
          [26]
          [51]
          [21]
          [19]
          [85]
          [12]
          [29]
          [30]]
      Transpose:
      [[ 7 21 12]
       [26 19 29]
       [51 85 30]]
      Flattened: [ 7 26 51 21 19 85 12 29 30]
      Add: [3 4 5 6 7]
      Multiply: [ 3 6 9 12 15]
      Square root: [1. 1.41421356 1.73205081 2. 2.23606798]
      Power: [ 1 4 9 16 25]
      Absolute: [2 3]
      Mean: 3.0
      Median: 3.0
      Standard Deviation: 1.4142135623730951
      Variance: 2.0
      Minimum: 1
      Maximum: 5
      Sum: 15
      Percentile (50): 3.0
      Column-wise sum:
      [ 40 74 166]
      Row-wise mean:
      [28. 41.66666667 23.66666667]
      First element: 1
      Slicer: [2 3 4]
      2D Slicer:
      [[26 51]
       [19 85]]
      Where > 50:
```

Variables Terminal

6:40 PM Python 3

colab.research.google.com

11249A276 - Colab

Gmail: Secure, AI-powered email for everyone... (no subject) - 11249a276@kanchiuniv.ac.in - ...

Start Page

11249A276

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

```
2D Slice:
[[26 51]
 [19 85]]
...
Where > 50:
(array([0, 1]), array([2, 2]))
Any > 90: False
All > 0: True
Sorted a: [1 2 3 4 5]
Index of max: 4
Index of min: 0
Unique elements: [1 2 3 4]
Dot Product:
[[19 22]
 [43 50]]
Matrix Multiplication:
[[19 22]
 [43 50]]
Determinant: -2.0000000000000004
Inverse:
[[-2.   1. ]
 [ 1.5 -0.5]]
Random choice: [4 5 1]
Concatenate: [1 2 3 4 5 6]
Vertical Stack:
[[1 2 3]
 [4 5 6]]
Horizontal Stack: [1 2 3 4 5 6]
```

Variables Terminal

6:40 PM Python 3

PANDAS

Aim:

To use Pandas for data manipulation and analysis, including tasks such as data cleaning, transformation, and handling structured datasets efficiently.

```
import pandas as pd
```

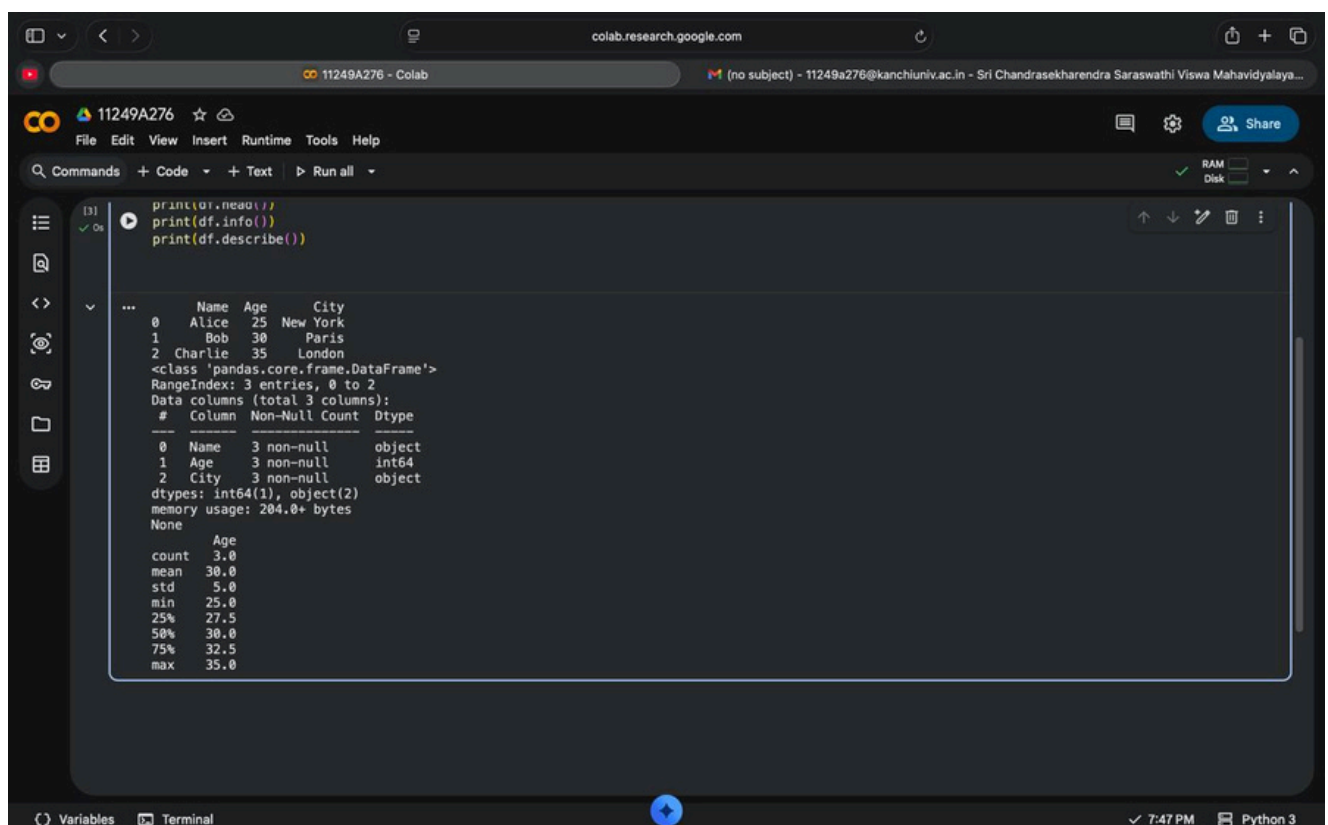
A) Creating DataFrame

```
data = {  
    "Name": ["Alice", "Bob", "Charlie"],  
    "Age": [25, 30, 35],  
    "City": ["New York", "Paris", "London"]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df.head())  
print(df.info())  
print(df.describe())
```

OUTPUT:



The screenshot shows a Google Colab notebook interface. The code cell contains the following Python code:

```
print(df.head())  
print(df.info())  
print(df.describe())
```

The output of the code is displayed below the code cell. It shows the first three rows of the DataFrame, the information about the DataFrame, and the descriptive statistics for the 'Age' column.

```
...  
   Name  Age  City  
0  Alice   25  New York  
1   Bob   30   Paris  
2  Charlie  35  London  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 3 entries, 0 to 2  
Data columns (total 3 columns):  
#   Column  Non-Null Count  Dtype  
---  ---  
0    Name    3 non-null        object  
1    Age      3 non-null        int64  
2    City     3 non-null        object  
dtypes: int64(1), object(2)  
memory usage: 204.0+ bytes  
None  
Age  
count    3.0  
mean     30.0  
std       5.0  
min      25.0  
25%      27.5  
50%      30.0  
75%      32.5  
max      35.0
```

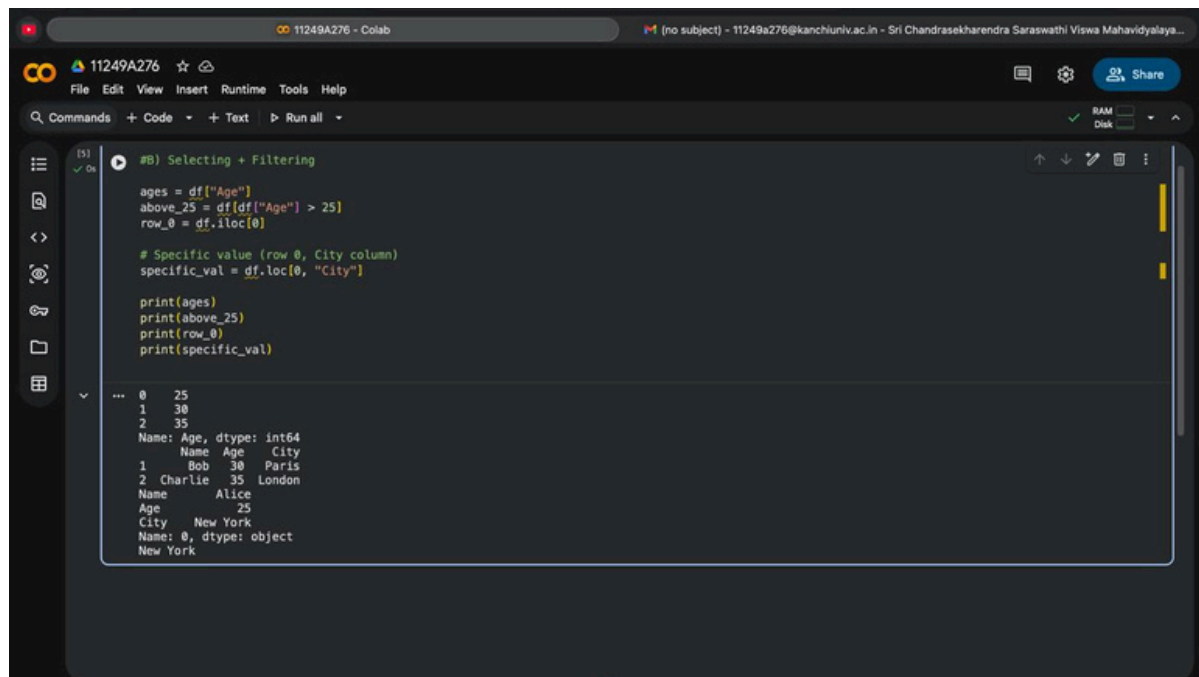
B) Selecting + Filtering

```
ages = df["Age"]  
above_25 = df[df["Age"] > 25]  
row_0 = df.iloc[0]
```

```
# Specific value (row 0, City column)  
specific_val = df.loc[0, "City"]
```

```
print(ages)  
print(above_25)  
print(row_0)  
print(specific_val)
```

OUTPUT:



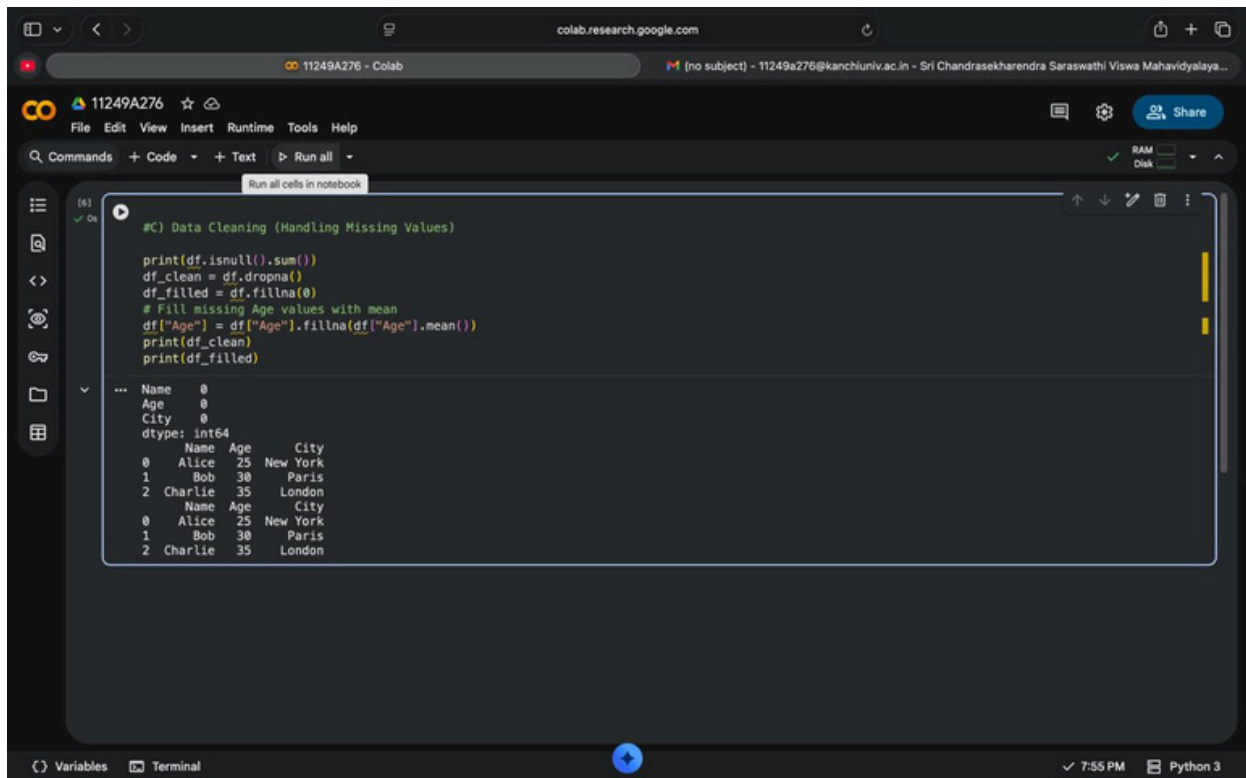
```
#B) Selecting + Filtering  
  
ages = df["Age"]  
above_25 = df[df["Age"] > 25]  
row_0 = df.iloc[0]  
  
# Specific value (row 0, City column)  
specific_val = df.loc[0, "City"]  
  
print(ages)  
print(above_25)  
print(row_0)  
print(specific_val)
```

```
0    25  
1    30  
2    35  
Name: Age, dtype: int64  
   Name  Age  City  
1   Bob   30  Paris  
2  Charlie 35  London  
Name      Alice  
Age         25  
City    New York  
Name: 0, dtype: object  
New York
```

C) Data Cleaning (Handling Missing Values)

```
print(df.isnull().sum())  
df_clean = df.dropna()  
df_filled = df.fillna(0)  
  
# Fill missing Age values with mean  
df["Age"] = df["Age"].fillna(df["Age"].mean())  
print(df_clean)  
print(df_filled)
```

OUTPUT:



The screenshot shows a Google Colab notebook interface. The browser address bar displays 'colab.research.google.com'. The notebook title is '11249A276 - Colab'. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The toolbar shows 'Commands', '+ Code', '+ Text', and 'Run all'. A 'Run all cells in notebook' button is visible. The code cell contains the following Python code:

```
#C) Data Cleaning (Handling Missing Values)

print(df.isnull().sum())
df_clean = df.dropna()
df_filled = df.fillna(0)
# Fill missing Age values with mean
df["Age"] = df["Age"].fillna(df["Age"].mean())
print(df_clean)
print(df_filled)
```

The output of the code is displayed below the cell:

```
... Name      0
Age      0
City      0
dtype: int64
   Name  Age  City
0  Alice  25 New York
1   Bob   30  Paris
2 Charlie  35  London
   Name  Age  City
0  Alice  25 New York
1   Bob   30  Paris
2 Charlie  35  London
```

The bottom status bar shows 'Variables', 'Terminal', '7:55 PM', and 'Python 3'.

MATPLOTLIB(plotting library)

Aim:

To use Matplotlib for creating visualizations such as line plots, bar charts, and scatter plots to analyze and interpret data effectively.

Program:

```
import pandas as pd

import matplotlib.pyplot as plt

import numpy as np

data = {

    "StudyHours": [1, 2, 3, 4, 5, 6, 7, 8],

    "ExamScore": [35, 40, 50, 55, 65, 70, 78, 85]

}

df = pd.DataFrame(data)
```

1) Scatter Plot

```
plt.scatter(df["StudyHours"], df["ExamScore"])

plt.xlabel("Study Hours")

plt.ylabel("Exam Score")

plt.title("Study Hours vs Exam Score (Scatter Plot)")
```

```
plt.show()
```

2) Line Plot

```
plt.plot(df["StudyHours"], df["ExamScore"], marker='o')
```

```
plt.xlabel("Study Hours")
```

```
plt.ylabel("Exam Score")
```

```
plt.title("Exam Score Progress (Line Plot)")
```

```
plt.show()
```

3) Histogram

```
plt.hist(df["ExamScore"], bins=5)
```

```
plt.xlabel("Score Range")
```

```
plt.ylabel("Number of Students")
```

```
plt.title("Exam Score Distribution (Histogram)")
```

```
plt.show()
```

4) Bar Chart

```
plt.bar(df["StudyHours"], df["ExamScore"])
```

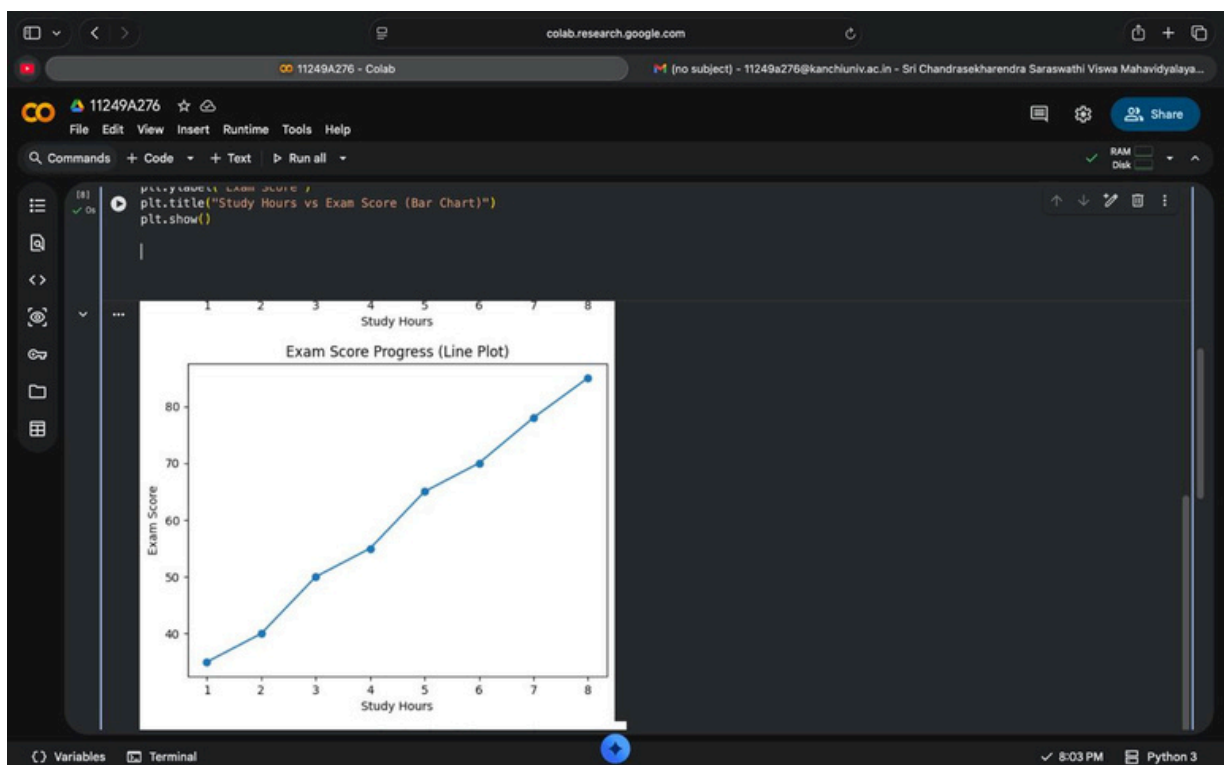
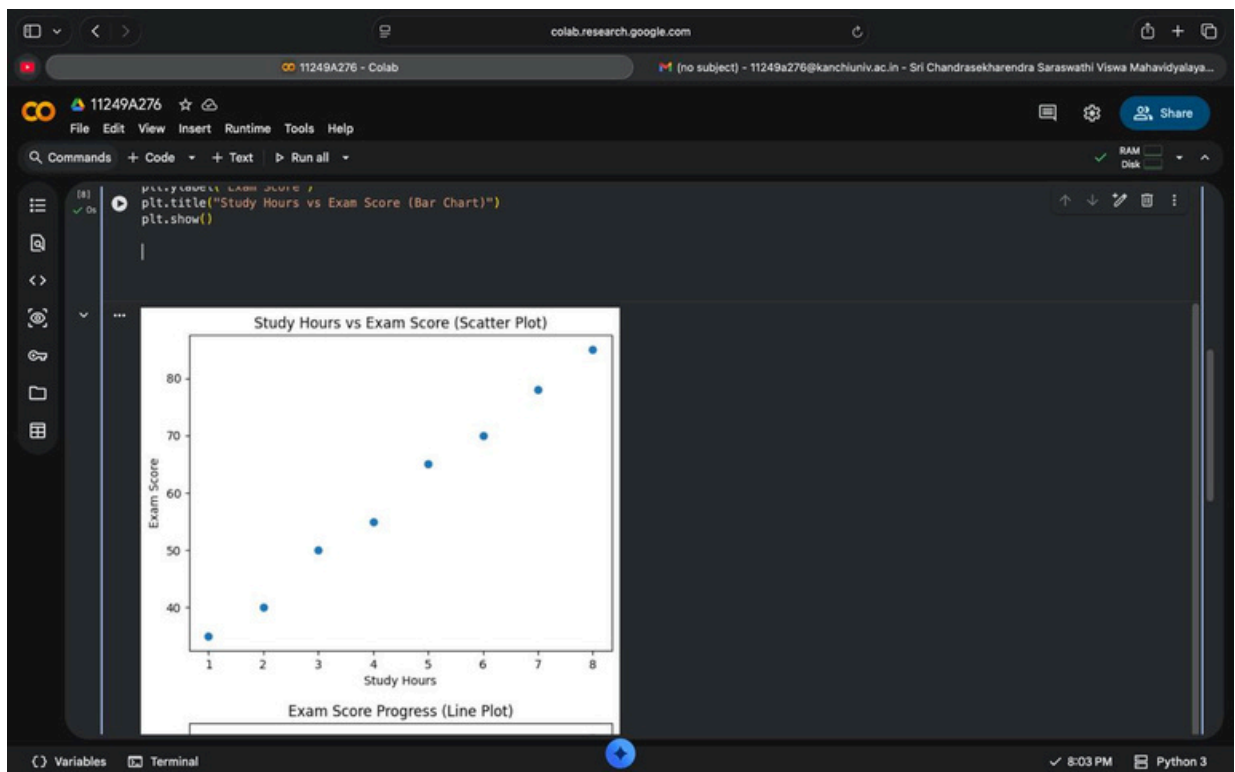
```
plt.xlabel("Study Hours")
```

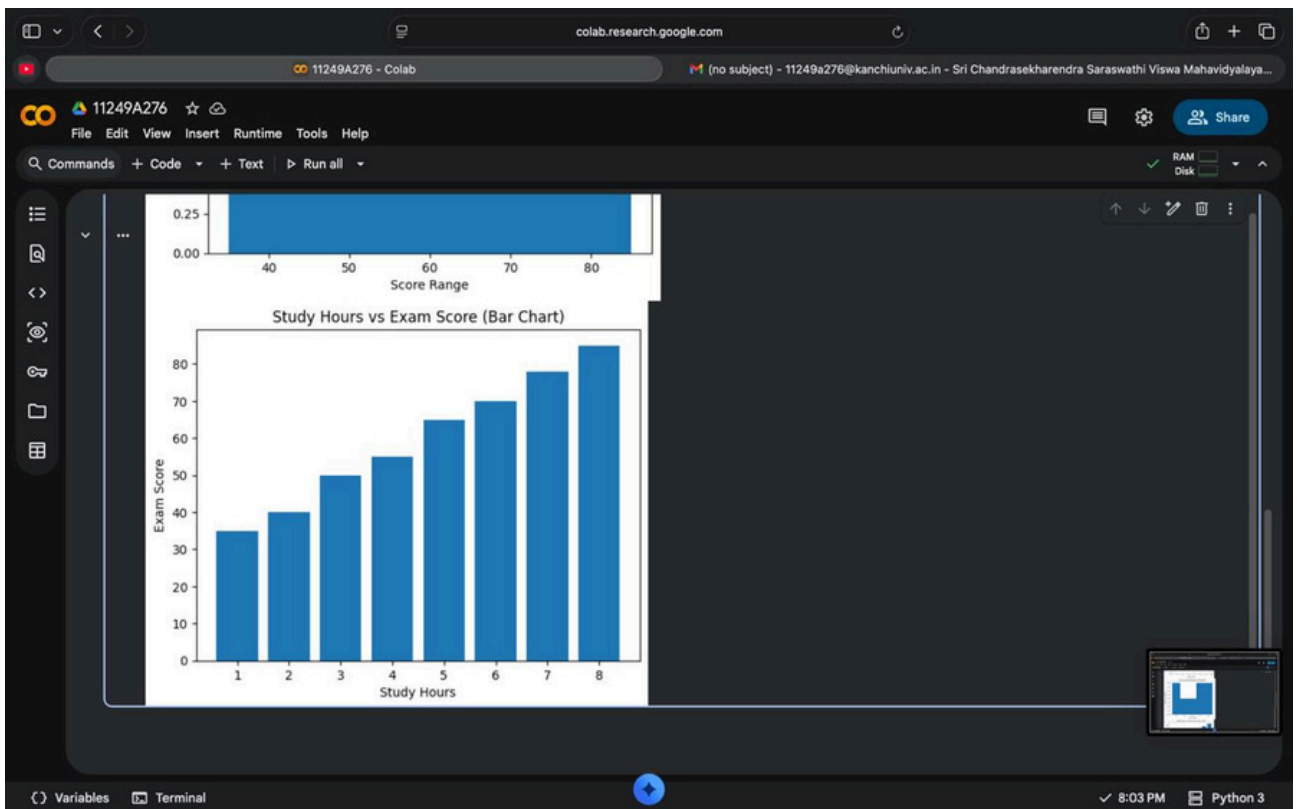
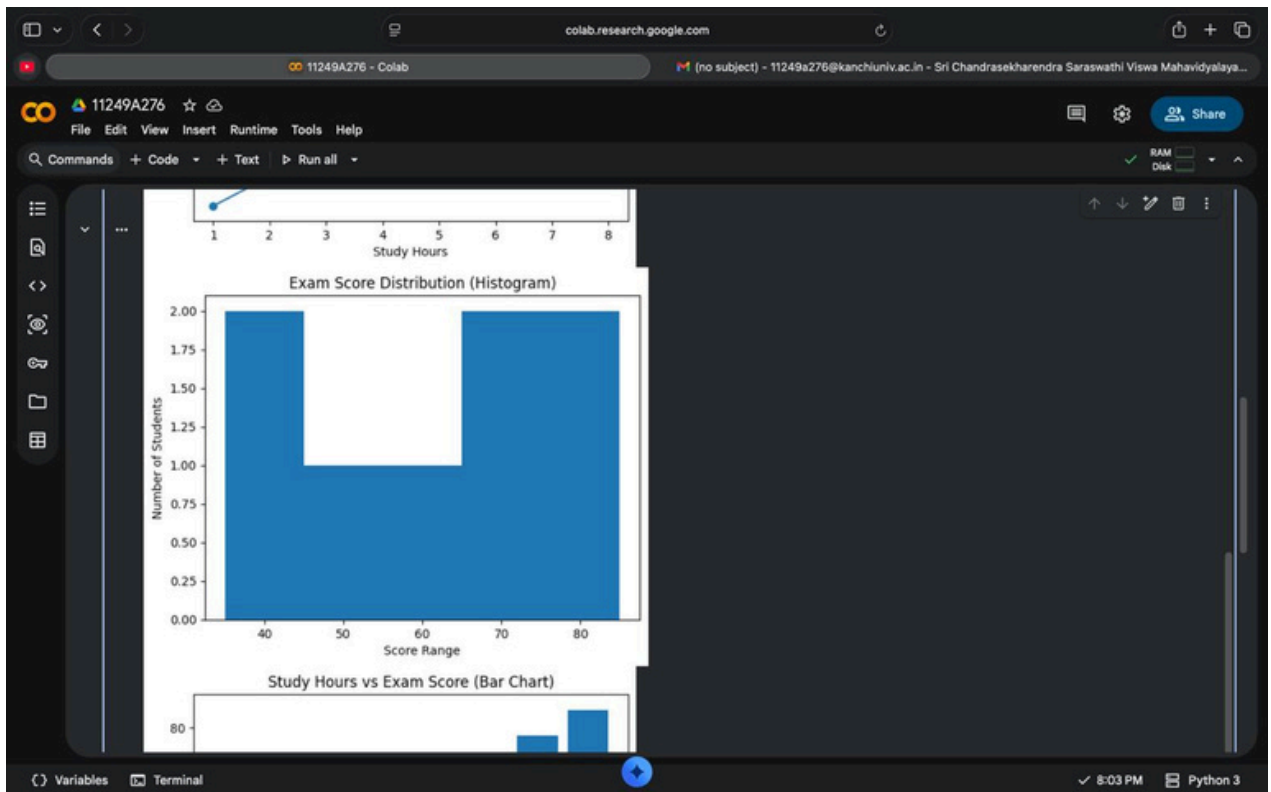
```
plt.ylabel("Exam Score")
```

```
plt.title("Study Hours vs Exam Score (Bar Chart)")
```

plt.show()

OUTPUT:





SCIKIT LEARN(MACHINE LEARNING)

Aim:

To study and implement fundamental machine learning algorithms using Scikit-learn for building, training, and evaluating predictive models on various datasets.

STEP 1: IMPORT LIBRARIES

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import mean_squared_error
```

STEP 2: CREATE DATASET

```
data = {  
  
    "Student": ["S1", "S2", "S3", "S4", "S5",  
  
    "S6", "S7", "S8", "S9", "S10"],  
  
    "Classes_Attended": [30, 35, 40, 45, 50,  
  
    55, 60, 65, 70, 75],  
  
    "Internal_Marks": [35, 49, 38, 46, 50,  
  
    55, 60, 65, 70, 75]  
}
```



```
df = pd.DataFrame(data)
```

```
print(df)
```

STEP 3: FEATURES + TARGET

```
X = df[["Classes_Attended"]] # Feature
```

```
y = df["Internal_Marks"] # Target
```

STEP 4: TRAIN - TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.2, random_state=7
```

```
)
```

STEP 5: TRAIN MODEL

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
print(model)
```

STEP 6: PREDICTION

```
predictions = model.predict(X_test)
```

STEP 7: EVALUATION

```
mse = mean_squared_error(y_test, predictions)
```

```
print("MSE:", mse)
```

```
print("Marks per Class:", model.coef_[0])
```

```
print("Base Marks:", model.intercept_)
```

STEP 8: VISUALIZATION

```
plt.scatter(X, y)
```

```
plt.plot(X, model.predict(X))
```

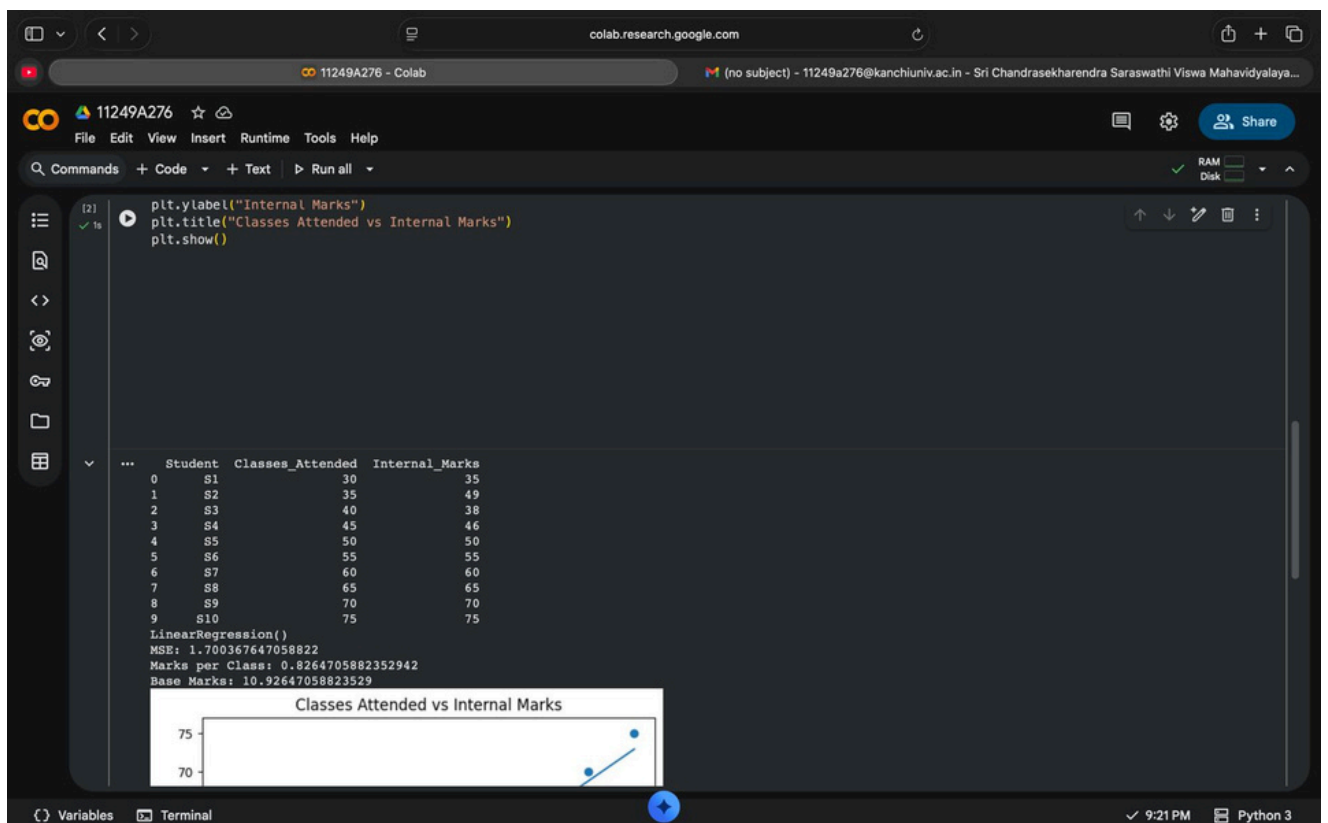
```
plt.xlabel("Classes Attended")
```

```
plt.ylabel("Internal Marks")
```

```
plt.title("Classes Attended vs Internal Marks")
```

```
plt.show()
```

OUTPUT:



colab.research.google.com

11249A276 - Colab

(no subject) - 11249a276@kanchiuniv.ac.in - Sri Chandrasekharendra Saraswathi Viswa Mahavidyalaya...

11249A276

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

...

```
7 s8 65 65
8 s9 70 70
9 s10 75 75
LinearRegression()
MSE: 1.700367647058822
Marks per Class: 0.8264705882352942
Base Marks: 10.92647058823529
```

Classes Attended vs Internal Marks

Classes Attended	Internal Marks
30	35
35	49
40	38
45	46
50	50
55	55
60	60
65	65
70	70
75	75

Variables Terminal

9:21 PM Python 3

EX:-02

PERFORM DATA EXPLORATION and PRE-PROCCESING IN PYTHON

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

# --- STEP 1: LOAD DATA (Creating a dummy dataset for the lab) ---

data = {

    'Country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],

    'Age': [22, 25, np.nan, 30, 28, 35], # Note the 'nan' (missing value)

    'Salary': [40000, 60000, 50000, np.nan, 72000, 58000],

    'Purchased': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']

}

df = pd.DataFrame(data)

print("--- ORIGINAL RAW DATA ---")

print(df)

print("\n")

# --- STEP 2: INSPECT DATA ---
```

```
# Check for missing values
```

```
print("--- MISSING VALUES COUNT ---")
```

```
print(df.isnull().sum())
```

```
print("\n")
```

```
# --- STEP 3: CLEAN DATA (Handling Missing Values) ---
```

```
# Logic: Fill missing Age/Salary with the Average (Mean) of that column
```

```
df['Age'] = df['Age'].fillna(df['Age'].mean())
```

```
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
```

```
print("--- DATA AFTER CLEANING ---")
```

```
print(df)
```

```
print("\n")
```

```
# --- STEP 4: CONVERT TEXT TO NUMBERS (Encoding) ---
```

```
# Logic: Machines can't read 'India' or 'USA'. We convert them to 0s and 1s.
```

```
# We use 'One Hot Encoding' (get_dummies)
```

```
df_encoded = pd.get_dummies(df, columns=['Country'])
```

```
# For the Target column (Purchased), let's map Yes/No manually
```

```
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})
```

```
print("--- DATA AFTER ENCODING (All Numbers Now) ---")
```

```
print(df_encoded)print("\n")
```

```
# --- STEP 5: SCALE FEATURES ---
```

Logic: Age is 20-30, Salary is 40000-60000. Salary dominates Age because it's bigger.

We shrink them to the same scale.

Separate Features (X) and Target (y)

```
X = df_encoded.drop('Purchased', axis=1)
```

```
y = df_encoded['Purchased']
```

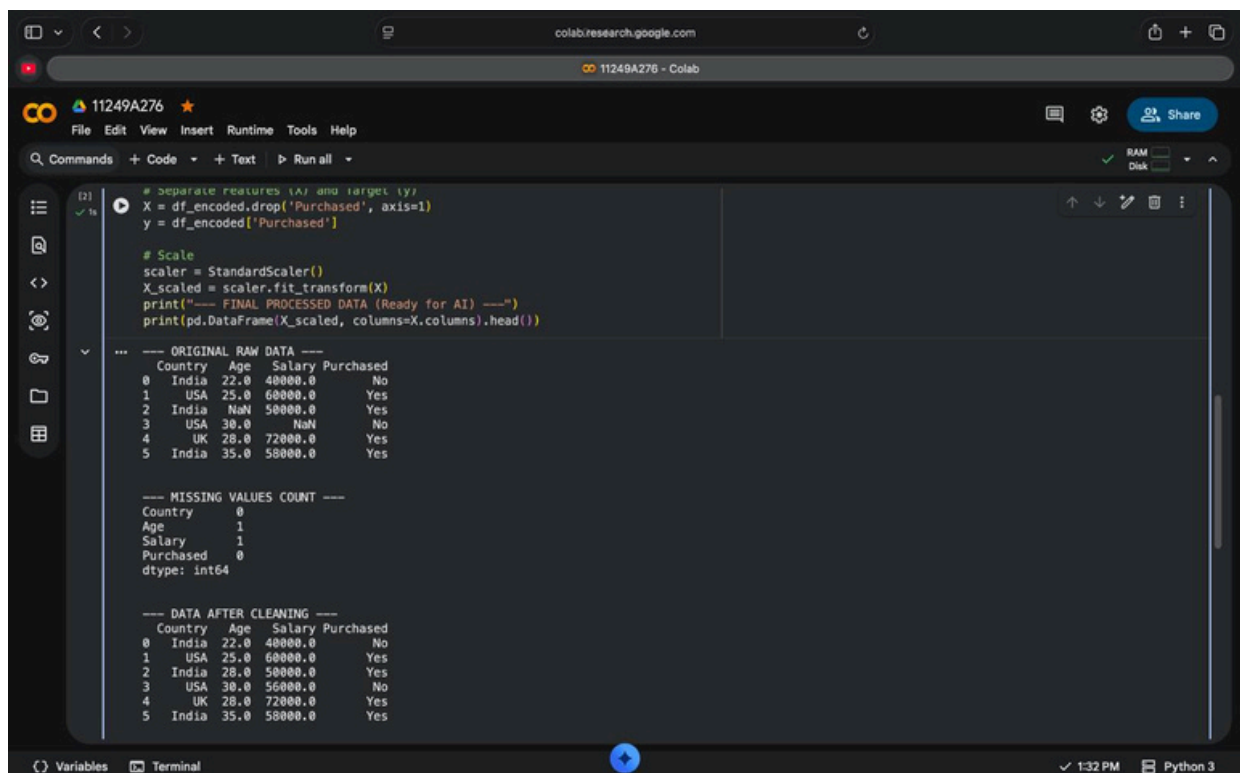
Scale

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
print("--- FINAL PROCESSED DATA (Ready for AI) ---")
```

```
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```



The screenshot shows a Google Colab notebook interface. The code cell contains the following Python code:

```
# Separate Features (X) and Target (y)
X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']

# Scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("--- FINAL PROCESSED DATA (Ready for AI) ---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```

The output of the code cell displays the original raw data, missing values count, and data after cleaning.

--- ORIGINAL RAW DATA ---

	Country	Age	Salary	Purchased
0	India	22.0	40000.0	No
1	USA	25.0	60000.0	Yes
2	India	NaN	50000.0	Yes
3	USA	30.0	NaN	No
4	UK	28.0	72000.0	Yes
5	India	35.0	50000.0	Yes

--- MISSING VALUES COUNT ---

```
Country    0
Age        1
Salary     1
Purchased  0
dtype: int64
```

--- DATA AFTER CLEANING ---

	Country	Age	Salary	Purchased
0	India	22.0	40000.0	No
1	USA	25.0	60000.0	Yes
2	India	28.0	50000.0	Yes
3	USA	30.0	50000.0	No
4	UK	28.0	72000.0	Yes
5	India	35.0	50000.0	Yes

colab.research.google.com

11249A276 - Colab

11249A276

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk

Country Age Salary Purchased

0 India 22.0 40000.0 No

1 USA 25.0 60000.0 Yes

2 India 28.0 50000.0 Yes

3 USA 30.0 56000.0 No

4 UK 28.0 72000.0 Yes

5 India 35.0 58000.0 Yes

--- DATA AFTER ENCODING (All Numbers Now) ---

Age Salary Purchased Country_India Country_UK Country_USA

0 22.0 40000.0 0 True False False

1 25.0 60000.0 1 False False True

2 28.0 50000.0 1 True False False

3 30.0 56000.0 0 False False True

4 28.0 72000.0 1 False True False

5 35.0 58000.0 1 True False False

--- FINAL PROCESSED DATA (Ready for AI) ---

Age Salary Country_India Country_UK Country_USA

0 -1.484615 -1.644453 1.0 -0.447214 -0.707107

1 -0.742307 0.411113 -1.0 -0.447214 1.414214

2 0.000000 -0.616670 1.0 -0.447214 -0.707107

3 0.494872 0.000000 -1.0 -0.447214 1.414214

4 0.000000 1.644453 -1.0 2.236068 -0.707107

Variables Terminal

1:32 PM Python 3

3.IMPLEMENT REGULARISED LINEAR REGRESSION

Aim:

To implement **regularized linear regression** using Scikit-learn in order to build a regression model while reducing overfitting through regularization techniques such as Ridge (L2) and Lasso (L1), and to evaluate its performance on a given dataset.

PROGRAM:

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
```

```
from sklearn.metrics import mean_squared_error
```

```
from sklearn.datasets import load_diabetes
```

```
from sklearn.model_selection import train_test_split
```

Load dataset

```
diabetes = load_diabetes()
```

Split data

```
x_train, x_test, y_train, y_test = train_test_split(
```

```
    diabetes.data,
```

```
    diabetes.target,
```

```
    test_size=0.2,
```

```
    random_state=42
```

```
)
```

Models


```
lr = LinearRegression().fit(x_train, y_train)
```

```
ridge = Ridge(alpha=10).fit(x_train, y_train)
```

```
lasso = Lasso(alpha=0.1).fit(x_train, y_train)
```

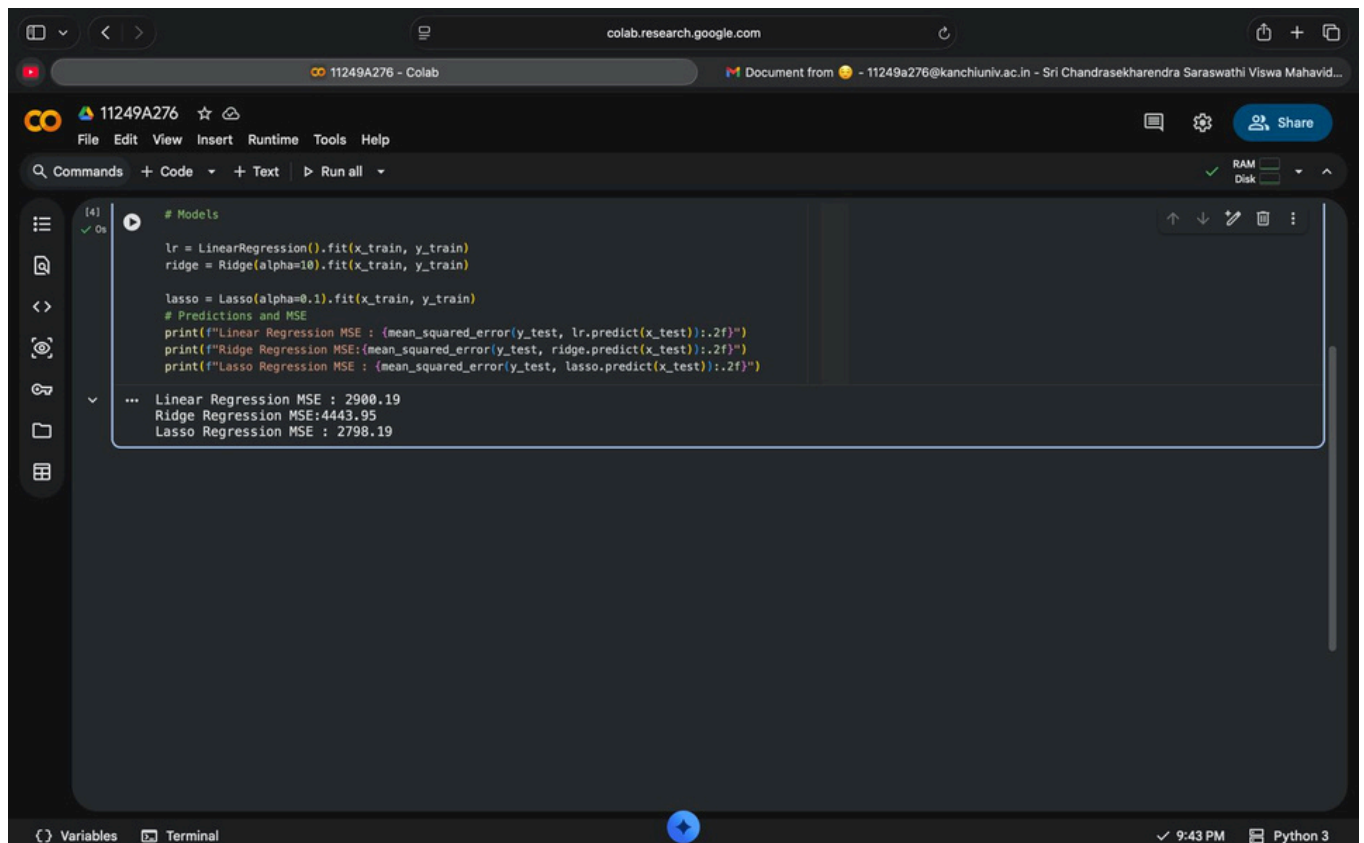
Predictions and MSE

```
print(f"Linear Regression MSE : {mean_squared_error(y_test,  
lr.predict(x_test)):.2f}")
```

```
print(f"Ridge Regression  
MSE:{mean_squared_error(y_test,ridge.predict(x_test)):.2f}")
```

```
print(f"Lasso Regression MSE : {mean_squared_error(y_test,  
lasso.predict(x_test)):.2f}")
```

OUTPUT:



The screenshot displays a Google Colab notebook interface. The code cell, labeled [4], contains the following Python code:

```
# Models  
lr = LinearRegression().fit(x_train, y_train)  
ridge = Ridge(alpha=10).fit(x_train, y_train)  
  
lasso = Lasso(alpha=0.1).fit(x_train, y_train)  
  
# Predictions and MSE  
print(f"Linear Regression MSE : {mean_squared_error(y_test, lr.predict(x_test)):.2f}")  
print(f"Ridge Regression MSE:{mean_squared_error(y_test, ridge.predict(x_test)):.2f}")  
print(f"Lasso Regression MSE : {mean_squared_error(y_test, lasso.predict(x_test)):.2f}")
```

The output of the code cell shows the MSE values for each model:

```
... Linear Regression MSE : 2900.19  
Ridge Regression MSE:4443.95  
Lasso Regression MSE : 2798.19
```

The bottom status bar of the Colab interface indicates the current time as 9:43 PM and the Python version as Python 3.

4.IMPLEMENT NAIVE BAYES CLASSIFIER

Aim:

To implement the **Naive Bayes classifier** using Scikit-learn for building a probabilistic classification model based on Bayes' theorem, and to evaluate its performance on a given dataset.

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score
```

Dataset

```
data = {  
  
    'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],  
  
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],  
  
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']  
  
}
```

Create DataFrame

```
df = pd.DataFrame(data)
```

Features and Target

```
X = df[['StudyHours', 'Attendance']]
```

```
Y = df['Result']
```

Split dataset

```
X_train, X_test, Y_train, Y_test = train_test_split(
```

```
    X, Y, test_size=0.25, random_state=42
```

```
)
```

Model

```
model = GaussianNB()
```

Train model

```
model.fit(X_train, Y_train)
```

Prediction

```
Y_pred = model.predict(X_test)
```

Accuracy

```
print("Predicted Output:", Y_pred)
```

```
print("Accuracy:", accuracy_score(Y_test, Y_pred))
```

OUTPUT:

colab.research.google.com

11249A276 - Colab

Document from 11249a276@kanchiuniv.ac.in - Sri Chandrasekharendra Saraswathi Viswa Mahavid...

11249A276

Saving...

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[5] ✓ Os

X, Y, test_size=0.25, random_state=42)

Model

model = GaussianNB()

Train model

model.fit(X_train, Y_train)

Prediction

Y_pred = model.predict(X_test)

Accuracy

print("Predicted Output:", Y_pred)

print("Accuracy:", accuracy_score(Y_test, Y_pred))

...

Predicted Output: ['Fail' 'Fail']

Accuracy: 1.0

Variables Terminal

9:49 PM Python 3

5.IMPLEMENT REGULARIZED LOGISTIC REGRESSION

Aim:

To implement **regularized logistic regression** using Scikit-learn in order to build a classification model while controlling overfitting through regularization techniques such as L1 and L2 penalties, and to evaluate its performance on a given dataset.

1. Import libraries

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score
```

2. Load Dataset

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

3. Split data

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

4. Scaling

```
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
  
X_test_scaled = scaler.transform(X_test)
```

5. Normal Logistic Regression (very large C)

```
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')  
  
normal_model.fit(X_train_scaled, y_train)  
  
normal_pred = normal_model.predict(X_test_scaled)  
  
normal_acc = accuracy_score(y_test, normal_pred)
```

6. Regularized Logistic Regression

```
regularized_model = LogisticRegression(C=0.1, penalty='l2',  
solver='liblinear')  
  
regularized_model.fit(X_train_scaled, y_train)  
  
reg_pred = regularized_model.predict(X_test_scaled)  
  
reg_acc = accuracy_score(y_test, reg_pred)
```

7. Print Results

```
print("Normal Logistic Accuracy:", normal_acc * 100)
```

```
print("Regularized Logistic Accuracy:", reg_acc * 100)
```

8. Compare Coefficients

```
plt.figure()
```

```
plt.plot(normal_model.coef_.flatten(), label='Normal Logistic')
```

```
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic')
```

```
plt.legend()
```

```
plt.title("Coefficient Comparison")
```

```
plt.xlabel("Feature Index")
```

```
plt.ylabel("Coefficient Value")
```

```
plt.show()
```

9. Accuracy Comparison Bar Graph

```
plt.figure()
```

```
models = ['Normal Logistic', 'Regularized Logistic']
```

```
accuracies = [normal_acc, reg_acc]
```

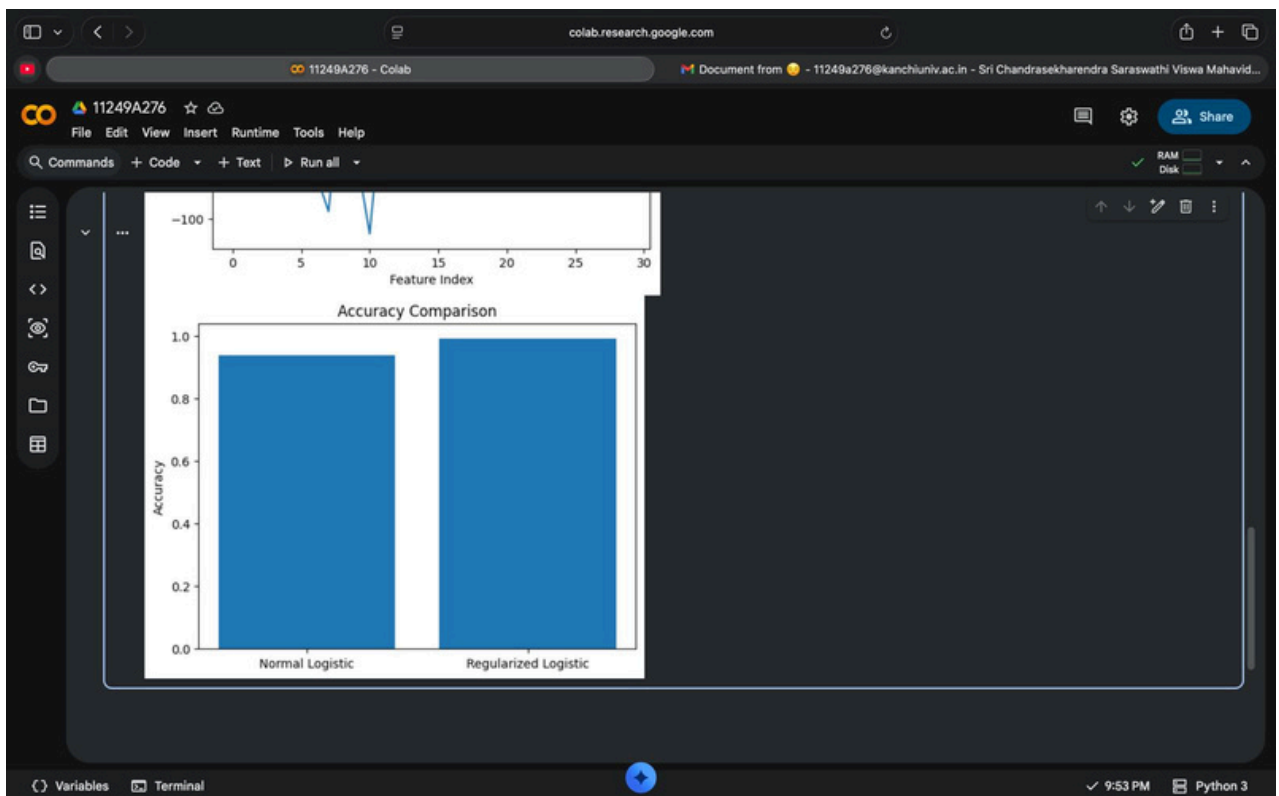
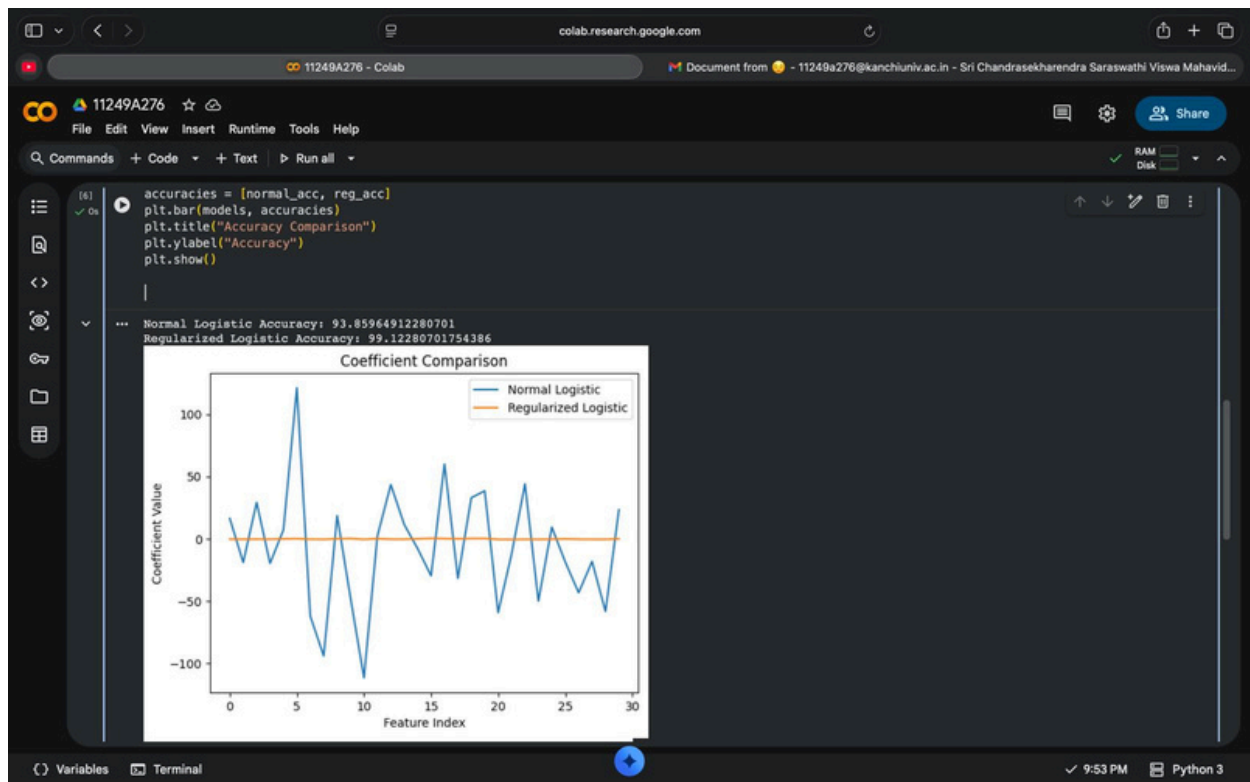
```
plt.bar(models, accuracies)
```

```
plt.title("Accuracy Comparison")
```

```
plt.ylabel("Accuracy")
```

```
plt.show()
```

OUTPUT:



6. Build models using different ensembling techniques

Aim:

To implement and compare different ensemble learning techniques using Scikit-learn—specifically Random Forest, AdaBoost, and Stacking classifiers—for building predictive models and evaluating their performance on a dataset.

```
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,  
StackingClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn import datasets
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

Load dataset

```
iris = datasets.load_iris()
```

Split data

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    iris.data, iris.target, test_size=0.2, random_state=42
```

```
)
```

Random Forest

```
rf = RandomForestClassifier(n_estimators=100)
```

```
rf.fit(X_train, y_train)
```

AdaBoost

```
ada = AdaBoostClassifier(n_estimators=100)
```

```
ada.fit(X_train, y_train)
```

Stacking

```
estimators = [
```

```
    ('rf', RandomForestClassifier(n_estimators=100)),
```

```
    ('svc', SVC(probability=True))
```

```
]
```

```
stack = StackingClassifier(
```

```
    estimators=estimators,
```

```
    final_estimator=LogisticRegression()
```

```
)
```

```
stack.fit(X_train, y_train)
```

Predictions & Accuracy

```
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
```

```
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
```

```
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

OUTPUT:

colab.research.google.com

11249A276 - Colab

Document from 11249a276@kanchiuniv.ac.in - Sri Chandrasekharendra Saraswathi Viswa Mahavid...

11249A276

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[9] 1s

```
# AdaBoost
ada = AdaBoostClassifier(n_estimators=100)
ada.fit(X_train, y_train)
# Stacking
estimators = [
    ('rf', RandomForestClassifier(n_estimators=100)),
    ('svc', SVC(probability=True))
]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)
stack.fit(X_train, y_train)
# Predictions & Accuracy
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

... Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Variables Terminal

10:01 PM Python 3

7.BUILDING MODELS USING DECISION TREES

Objective:

To implement the Decision Tree algorithm for classification and regression.
from sklearn.tree
import DecisionTreeClassifier

PROGRAM:

```
from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score

iris = datasets.load_iris()

X_train, X_test, y_train, y_test = train_test_split(

    iris.data, iris.target, test_size=0.2, random_state=42

)

dt_clf = DecisionTreeClassifier(max_depth=3)

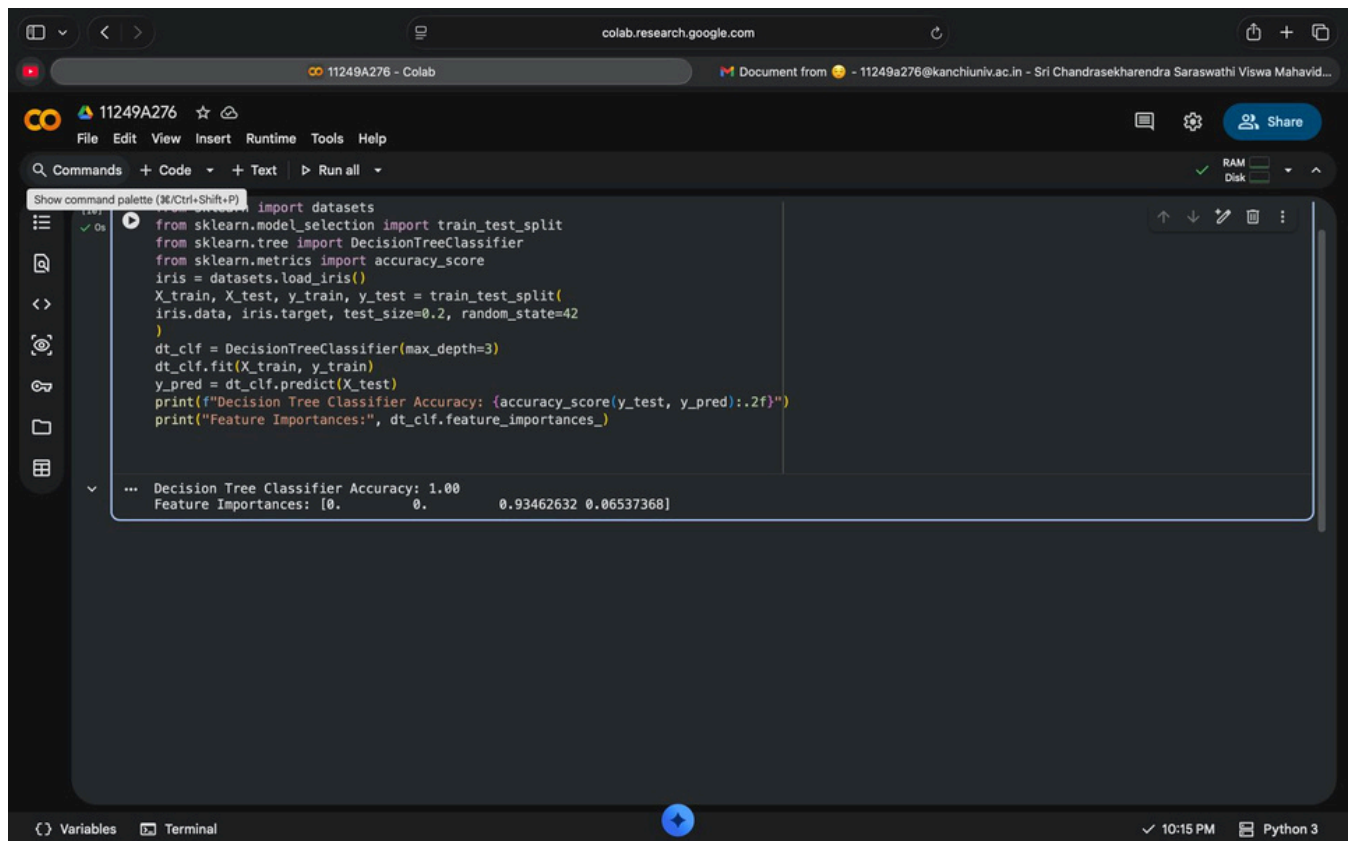
dt_clf.fit(X_train, y_train)

y_pred = dt_clf.predict(X_test)

print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, y_pred):.2f}")

print("Feature Importances:", dt_clf.feature_importances_)
```

OUTPUT:



The screenshot displays a Google Colab notebook interface. The top bar shows the URL 'colab.research.google.com' and a document title 'Document from 11249a276@kanchiuniv.ac.in - Sri Chandrasekharendra Saraswathi Viswa Mahavid...'. The notebook's menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The left sidebar contains icons for file management, search, and other functions. The main code editor area shows the following Python code:

```
import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)
dt_clf = DecisionTreeClassifier(max_depth=3)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

The output of the code is displayed in the bottom panel, showing the accuracy and feature importances:

```
... Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.          0.          0.93462632 0.06537368]
```

The bottom status bar indicates the notebook is running Python 3 and shows the time as 10:15 PM.

8) Support Vector Machine (SVM) with Different Kernels

Support Vector Machine is a supervised machine learning algorithm used for classification and regression problems.

SVM finds the best hyperplane that separates data points of different classes, Example:

Class A • Class B

SVM finds a line (in 2D) or hyperplane (in higher dimensions) that separates these classes with maximum margin.

Key terms:

Hyperplane

Decision boundary separating classes.

Support Vectors

The data points closest to the decision boundary.

Margin

Distance between hyperplane and nearest data points.

Kernel Functions

separate classes,

Sometimes data is not linearly separable. Kernels transform data into a higher dimension to

Common kernels:

1. Linear Kernel

Used v/hen data is linearly separable,

2 Polynomial Kernel

Creates curved decision boundaries,

3 RBF Kernel (Radial Basis Function)

Most commonly used kernel.

Algorithm Steps

1 Load dataset

- 2 Split dataset into training and testing
- 3 Train SVM model with different kernels
- 4 Predict output
- 5 Compare accuracy

Coding:

Import libraries

import pandas as pa

from sklearn. model selection import train test split from sklearn.svm import SVC

from sklearn metrics import accuracy score from sklearn.datasets import load iris

Load dataset

data = load iris()

X = data.data

y = data.target

Split dataset

X_train, X_test, y_train, y_test = train test split(X, y, test size=0.2)

Linear Kernel

model_linear = SVC(kernel='linear')

model_linear.fit(X_train, y_train)

pred_linear = model_linear.predict(X_test)

acc_linear = accuracy score(y_test, pred_linear)

print('Linear Kernel Accuracy:', acc_linear)

Polynomial Kernel

model_poly = SVC(kernel='poly', degree=3)

model_poly.fit(X_train, y_train)

pred_poly = model_poly.predict(X_test) acc_poly = accuracy score(y_test, pred_poly)

print('Polynomial Kernel Accuracy:', acc_poly)

RBF Kernel

model_rbf = SVC(kernel='rbf')

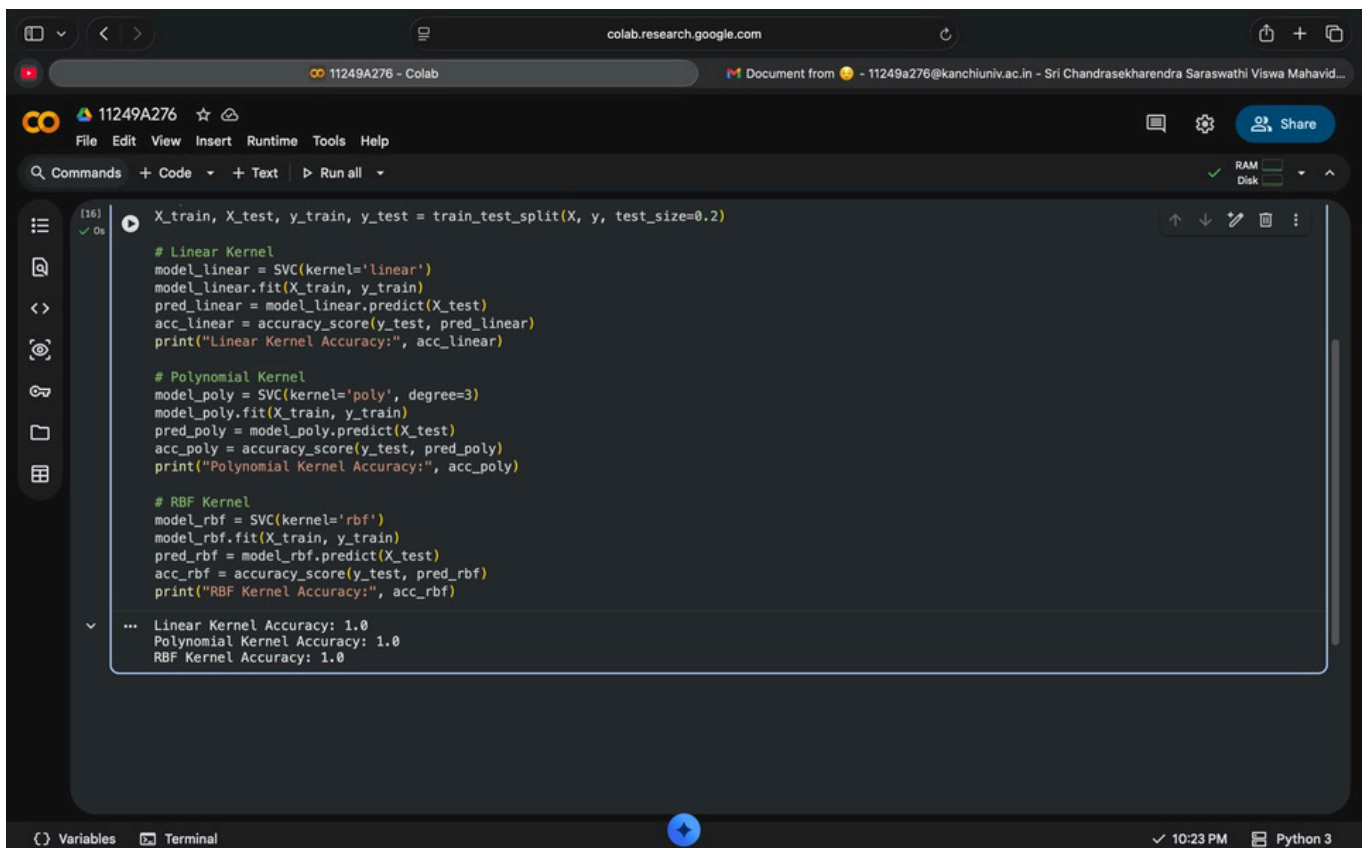
model_rbf.fit(X_train, y_train)

pred_rbf = model_rbf.predict(X_test)

acc_rbf = accuracy_score(y_test, pred_rbf)

print("RBF Kernel Accuracy:", acc_rbf)

Output:



The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, the notebook ID '11249A276', and a 'Share' button. The menu bar contains 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a search bar and a 'Run all' button. The left sidebar shows icons for file explorer, search, and other tools. The main code area contains the following Python code:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Linear Kernel
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)

# Polynomial Kernel
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)

# RBF Kernel
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)
```

The output of the code is displayed in a collapsed state, showing the following accuracies:

```
Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 1.0
RBF Kernel Accuracy: 1.0
```

The bottom status bar shows 'Variables', 'Terminal', a refresh button, the time '10:23 PM', and the Python version 'Python 3'.

9) Implement K-Nearest Neighbors (KNN) Classification

To implement the K-Nearest Neighbors (KNN) algorithm to classify mobile phones into price categories using Python.

Dataset:

Mobile Price Classification Dataset. It classifies mobile phones into 4 price ranges based on features like RAM, battery power, etc.

Kaggle Dataset:

This dataset is good because:

- Only numerical features (easy for KNN)
- Multi-class classification
- Clean CSV dataset suitable for engineering lab experiments

Algorithm

1. Import required libraries.
2. Load the dataset.
3. Split dataset into features (X) and target (y).
4. Divide data into training and testing sets.
5. Normalize features using StandardScaler.
6. Train the KNN classifier.
7. Predict results for test data.
8. Evaluate accuracy.

Coding:

Step 1: Import libraries

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Load dataset

```
data = pd.read_csv("mobile_price.csv")
```

Step 3: Define features and target

```
X = data.drop("price_range", axis=1)
```

```
y = data["price_range"]
```

Step 4: Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

Step 5: Feature scaling

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

Step 6: Create KNN model

```
knn = KNeighborsClassifier(n_neighbors=5)
```

Step 7: Train model

```
knn.fit(X_train, y_train)
```

Step 8: Prediction

```
y_pred = knn.predict(X_test)
```

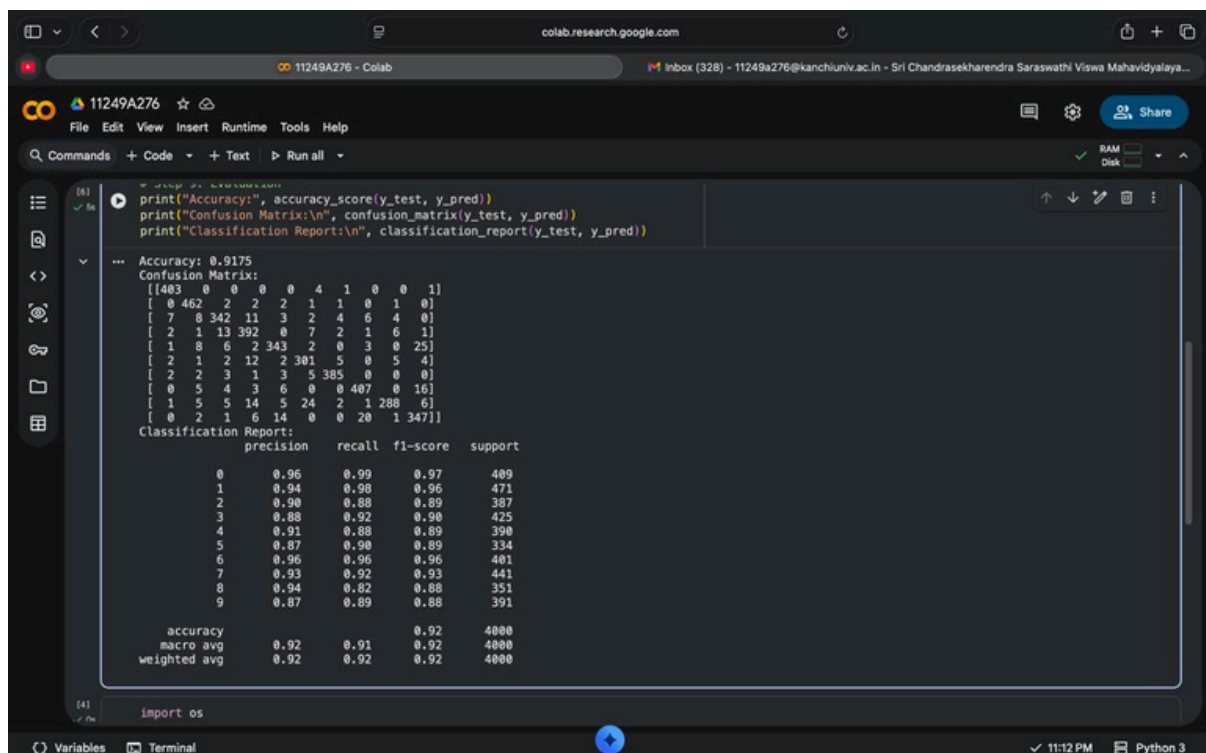
Step 9: Evaluation

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
print("Classification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:



```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9175
Confusion Matrix:
[[483  0  0  0  0  4  1  0  0  1]
 [ 0 462  2  2  2  1  1  0  1  0]
 [ 7  0 342 11  3  2  4  6  4  0]
 [ 2  1 13 392  0  7  2  1  6  1]
 [ 1  8  6  2 343  2  0  3  0 25]
 [ 2  1  2 12  2 301  5  0  5  4]
 [ 2  2  3  1  3  5 385  0  0  0]
 [ 0  5  4  3  6  0  0 407  0 16]
 [ 1  5  5 14  5 24  2  1 288  6]
 [ 0  2  1  6 14  0  0 20  1 347]]

Classification Report:
              precision    recall  f1-score   support

0             0.96         0.99         0.97         489
1             0.94         0.98         0.96         471
2             0.90         0.88         0.89         387
3             0.88         0.92         0.90         425
4             0.91         0.88         0.89         390
5             0.87         0.90         0.89         334
6             0.96         0.96         0.96         401
7             0.93         0.92         0.93         441
8             0.94         0.82         0.88         351
9             0.87         0.89         0.88         391

accuracy          0.92         0.92         0.92         4000
macro avg         0.92         0.91         0.92         4000
weighted avg      0.92         0.92         0.92         4000
```

10.) K-Means Clustering with PCA and Elbow Method

To perform clustering using K-Means algorithm after applying PCA (Principal Component Analysis) and determine the optimal K value using the Elbow Method.

Kaggle Dataset

Mall Customer Segmentation Dataset

- Dataset: Mall Customers

- Records: 200 customers

- Features:

- CustomerID

- o Gender

- Age

- Annual Income (k\$)

- Spending Score (1-100)

This dataset is widely used for customer segmentation using K-Means clustering and is ideal for student labs.

Typical clustering:

- High income - high spending

- High income - low spending

- Low income - high spending

- Low income - low spending

PROGRAM :

Step 1: Import libraries

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.cluster import KMeans
```

Step 2: Load dataset

```
data = pd.read_csv("Mall_Customers.csv")
```

Step 3: Select numerical features

```
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
```

Step 4: Feature Scaling

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Step 5: Apply PCA

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
```

```
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

Step 6: Elbow Method

```
wcss = [ ]
```

```
for i in range(1, 11):  
  
    kmeans = KMeans(n_clusters=i, random_state=0)  
  
    kmeans.fit(X_pca)  
  
    wcss.append(kmeans.inertia_)
```

Step 7: Plot Elbow Graph

```
plt.plot(range(1, 11), wcss, marker='o')  
  
plt.title("Elbow Method")  
  
plt.xlabel("Number of Clusters (K)")  
  
plt.ylabel("WCSS")  
  
plt.show()
```

Step 8: Apply KMeans with optimal K (e.g., K=5)

```
kmeans = KMeans(n_clusters=5, random_state=0)  
  
clusters = kmeans.fit_predict(X_pca)
```

Step 9: Plot Clusters

```
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')  
  
plt.title("K-Means Clustering with PCA")  
  
plt.xlabel("PCA Component 1")  
  
plt.ylabel("PCA Component 2")  
  
plt.show()
```

OUTPUT:

